



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 10

NOMBRE COMPLETO: Rosas Martinez Francisco Javier

N° de Cuenta: 320109766

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 8/Noviembre/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio 1:

Animar algún objeto por medio de Keyframes (que no sea ninguno de los personajes principales ni humanoide) requiere compartir capturas de pantalla donde se vea la impresión de los keyframes guardados o reproducidos en consola y guardados en un archivo de tipo txt y compartir los keyframes para que yo pueda reproducir su animación. La única restricción es que la Animación debe de representar el movimiento de un cuerpo en la vida real lógico, por ejemplo: una caja que va rodando no es un movimiento lógico. Un helado flotando no es un movimiento lógico. Debe de presentarse al objeto animado ya integrado en su proyecto final.

Para resolver el ejercicio, tuve que separar mi modelo de megacaja en dos para poder hacer que gire y rote a la vez, simulando como se abre la caja en el juego, en el código tuve que agregar las variables necesarias para ir añadiendo los keyframes. Por último en la ejecución del código fui añadiendo cada frame para lograr hacer mi animación de forma correcta.

Bloques de código generado

```
void AgregarKeyframeEnArchivo(const char* fileLocation, float x, float y, float z, float giro)
{
    std::ofstream fileStream(fileLocation, std::ios::app);

    if (!fileStream.is_open()) {
        printf("Error: No se pudo abrir el archivo %s para agregar un keyframe.\n", fileLocation);
        return;
    }

    // Escribe los valores
    fileStream << std::fixed << std::setprecision(1)
        << x << " " << y << " " << z << " " << giro << "\n";

    fileStream.close();
    printf("Keyframe agregado al archivo: (%.1f, %.1f, %.1f, %.1f)\n", x, y, z, giro);
}

void saveFrame(FRAME* KeyFrame, int& FrameIndex, float& mov_x, float& mov_y, float& mov_z, float& giro) //tecla L
{
    printf("frameindex %d\n", FrameIndex);

    KeyFrame[FrameIndex].mov_x = mov_x;
    KeyFrame[FrameIndex].mov_y = mov_y;
    KeyFrame[FrameIndex].mov_z = mov_z;
    KeyFrame[FrameIndex].giro = giro;

    // Guardar en el archivo de texto
    AgregarKeyframeEnArchivo("keyframes_megacaja.txt", mov_x, mov_y, mov_z, giro);

    FrameIndex++;
}
```

```

void animate(FRAME* KeyFrame, int& playIndex, int& i_curr_steps, int i_max_steps, int FrameIndex, float& mov_x, float& mov_y, float& mov_z, float& giro)
{
    if (play)
    {
        if (i_curr_steps >= i_max_steps) // Fin de animación entre frames
        {
            playIndex++;
            if (playIndex > FrameIndex - 2) // llegó al último frame
            {
                playIndex = 0; // reinicia desde el primer keyframe
            }

            i_curr_steps = 0; // resetea contador
            interpolation(KeyFrame, playIndex, i_max_steps); // calcula incrementos del nuevo tramo
        }
        else
        {
            // Interpolación movimiento
            mov_x += KeyFrame[playIndex].mov_xInc;
            mov_y += KeyFrame[playIndex].mov_yInc;
            mov_z += KeyFrame[playIndex].mov_zInc;
            giro += KeyFrame[playIndex].giroInc;
            i_curr_steps++;
        }
    }
}

```

```

void CargarKeyframesDesdeArchivo(const char* fileLocation, float& mov_x, float& mov_y, float& mov_z, float& giro, FRAME* KeyFrame, int& FrameIndex)
{
    Shader shader;
    std::string data = shader.ReadFile(fileLocation);

    if (data.empty()) {
        printf("No se pudieron cargar los keyframes desde %s\n", fileLocation);
        return;
    }

    std::stringstream stream(data);
    std::string line;
    int frameCount = 0;

    // Leer línea por línea
    while (std::getline(stream, line))
    {
        // Saltar encabezado
        if (line.empty() || line[0] == '#') continue;
        if (line.find("movAvion") != std::string::npos) continue;

        std::stringstream linestream(line);
        float x, y, z, giro;

        if (linestream >> x >> y >> z >> giro)
        {
            KeyFrame[frameCount].mov_x = x;
            KeyFrame[frameCount].mov_y = y;
            KeyFrame[frameCount].mov_z = z;
            KeyFrame[frameCount].giro = giro;
            frameCount++;
        }
    }

    FrameIndex = frameCount;
    printf("Se cargaron %d keyframes desde %s\n", frameCount, fileLocation);

    if (FrameIndex > 1) {
        resetElements(KeyFrame, mov_x, mov_y, mov_z, giro);
        interpolation(KeyFrame, playIndex, i_max_steps);
        play = true;
        playIndex = 0;
        i_curr_steps = 0;
    }
}

```

```

FRAME KeyFrameMegacaja[MAX_FRAMES];
int FrameIndexMegacaja = 0;
int playIndexMegacaja = 0;
int i_max_stepsMegacaja = 100;
int i_curr_stepsMegacaja = 0;

//NUEVOS Keyframes
float posXMegacaja = 0.0f, posYMegacaja = 0.0f, posZMegacaja = 0.0f;
float movMegacaja_x = 0.0f, movMegacaja_y = 0.0f, movMegacaja_z = 0.0f;
float giroMegacaja = 0.0f;

```

```

FRAME KeyFrameMegacaja2[MAX_FRAMES];
int FrameIndexMegacaja2 = 0;
int playIndexMegacaja2 = 0;
int i_max_stepsMegacaja2 = 100;
int i_curr_stepsMegacaja2 = 0;

//NEW// Keyframes
float posXmegacaja2 = 0.0f, posYmegacaja2 = 0.0f, posZmegacaja2 = 0.0f;
float movMegacaja2_x = 0.0f, movMegacaja2_y = 0.0f, movMegacaja2_z = 0.0f;
float giroMegacaja2 = 0.0f;

```

```

// Mega caja key frames
bs_megacajakey1 = Model();
bs_megacajakey1.LoadModel("Models/Brawl_stars/bs_megacajakey.obj");

```

```

CargarKeyframesDesdeArchivo("keyframes_megacaja.txt", movMegacaja_x, movMegacaja_y, movMegacaja_z, giroMegacaja, KeyFrameMegacaja, FrameIndexMegacaja);
CargarKeyframesDesdeArchivo("keyframes_megacaja2.txt", movMegacaja2_x, movMegacaja2_y, movMegacaja2_z, giroMegacaja2, KeyFrameMegacaja2, FrameIndexMegacaja2);

```

```

// Para la megacaja key
glm::mat4 megacajaAux(1.0);
glm::vec3 megacajaPos(0.0f, 0.0f, 0.0f);

```

```

//InputKeyframes(KeyFrameMegacaja, mainWindow.getKeys(), movMegacaja_x, movMegacaja_y, movMegacaja_z, giroMegacaja);
animate(KeyFrameMegacaja, playIndexMegacaja, i_curr_stepsMegacaja, i_max_stepsMegacaja, FrameIndexMegacaja, movMegacaja_x, movMegacaja_y, movMegacaja_z, giroMegacaja);

animate(KeyFrameMegacaja2, playIndexMegacaja2, i_curr_stepsMegacaja2, i_max_stepsMegacaja2, FrameIndexMegacaja2, movMegacaja2_x, movMegacaja2_y, movMegacaja2_z, giroMegacaja2);

```

```

// Mega caja key
model = ringCentro;
model = glm::translate(model, glm::vec3(-100.0f, 1.5f, -35.0f));
megacajaPos = glm::vec3(posXmegacaja + movMegacaja_x, posYmegacaja + movMegacaja_y * 0.1, posZmegacaja + movMegacaja_z);
model = glm::translate(model, megacajaPos);

// Escalado
model = glm::scale(model, glm::vec3(3.0f, 3.0f, 3.0f));

// Rotación
model = glm::rotate(model, giroMegacaja * 4 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
megacajaAux = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
bs_megacajakey1.RenderModel();

//megacaja 2
model = megacajaAux;
model = glm::translate(model, glm::vec3(-1.1f, 0.7f, 0.0f));
model = glm::rotate(model, giroMegacaja2 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
megacaja1.RenderModel();

```

Capturas del guardado de keyframes en archivo txt.

```

Presiona 6 para desbloquear el eje Y
Eje Y desbloqueado, puedes volver a mover con 4 (+Y) o 5 (-Y)

mov_y: 30.00 (+Y)
Presiona 6 para desbloquear el eje Y
Eje Y desbloqueado, puedes volver a mover con 4 (+Y) o 5 (-Y)
frameindex 0
Keyframe agregado al archivo: (0.0, 30.0, 0.0, 0.0)
mov_x es: 0.000000
mov_y es: 30.000000
mov_z es: 0.000000
presiona P para habilitar guardar otro frame'
Ya puedes guardar otro frame presionando la tecla L'

```

```

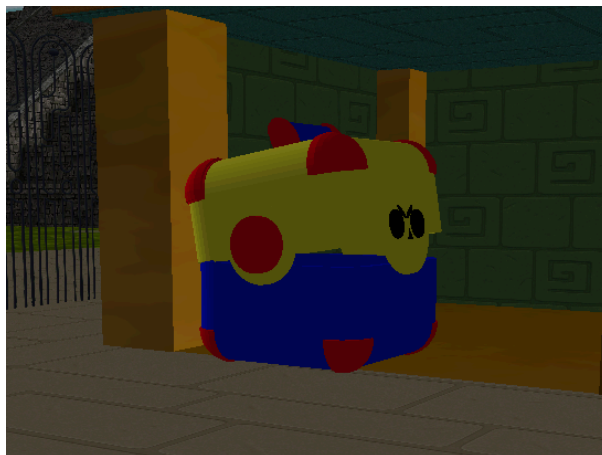
Giro: 60.00 grados (rotacion +Y)
Rotacion bloqueada. Presiona R para volver a habilitar.
Rotacion habilitada nuevamente. Puedes usar Q o E.
frameindex 0
Keyframe agregado al archivo: (0.0, 30.0, 0.0, 60.0)
mov_x es: 0.000000
mov_y es: 30.000000
mov_z es: 0.000000
presiona P para habilitar guardar otro frame'
Ya puedes guardar otro frame presionando la tecla L'

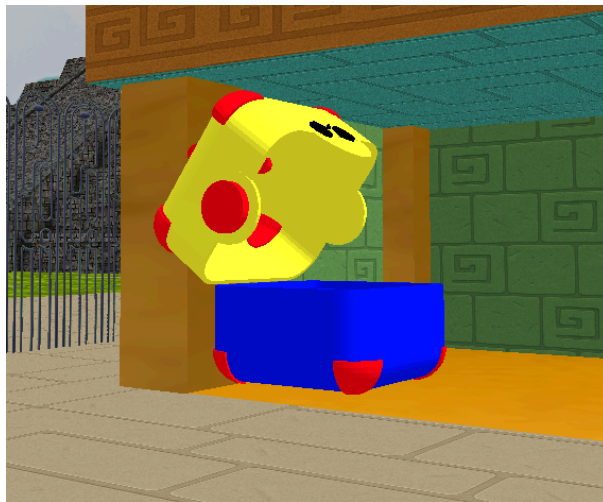
```

```
Giro: 180.00 grados (rotacion +Y)
Rotacion bloqueada. Presiona R para volver a habilitar
Rotacion habilitada nuevamente. Puedes usar Q o E.
frameindex 0
Keyframe agregado al archivo: (0.0, 30.0, 0.0, 180.0)
mov_x es: 0.000000
mov_y es: 30.000000
mov_z es: 0.000000
presiona P para habilitar guardar otro frame'
Ya puedes guardar otro frame presionando la tecla L'
|
```

```
Giro: 30.00 grados (rotacion +Y)
Rotacion bloqueada. Presiona R para volver a habilitar.
Rotacion habilitada nuevamente. Puedes usar Q o E.
frameindex 0
Keyframe agregado al archivo: (0.0, 0.0, 0.0, 30.0)
mov_x es: 0.000000
mov_y es: 0.000000
mov_z es: 0.000000
presiona P para habilitar guardar otro frame'
Ya puedes guardar otro frame presionando la tecla L'
|
```

Código en ejecución





2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

- Con esta práctica afortunadamente no tuve inconvenientes.

Conclusión:

Con esta práctica logré crear una animación que se repite de forma cíclica con un objeto relacionado al universo que elegí para mi proyecto final.

Con esto terminé todas las actividades que tenía que hacer para el proyecto. La materia me pareció muy interesante y, aunque la computación gráfica no era mi fuerte, pude completar cada práctica de manera bastante decente.

Bibliografía :

OpenGL Architecture Review Board, Shreiner, D., Woo, M., Neider, J., & Davis, T. (2010). OpenGL Programming Guide: The Official Guide to Learning OpenGL (7th ed.). Addison-Wesley.

OpenGL. (s. f.). OpenGL Documentation. Khronos Group. Recuperado de <https://www.khronos.org/opengl/>

Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3 (8th ed.). Addison-Wesley Professional.